Module 2 Note: React Hooks & Custom Hooks

useState, Rules of Hooks, Data Fetching, Advanced Hooks & Custom Hooks

Topic: Web Development / React **Document Version:** 1.0

01 The useState Hook

The `useState` hook allows functional components to maintain internal state across re-renders. It returns an array containing the current state value and an updater function.

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(prev => prev + 1)}>
      Count: {count}
    </button>
  );
}
```

02 Rules of Hooks & Fetching Data

> **The Two Fundamental Rules of Hooks**
>
> 1. **Only Call Hooks at the Top Level:** Do not call Hooks inside loops, conditions, or nested functions.
> 2. **Only Call Hooks from React Functions:** Call them from React functional components or custom Hooks.

Data fetching is handled side-by-side using `useEffect` alongside state management for loading, error, and data payload states.

```
import React, { useState, useEffect } from 'react';

function DataFetcher() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('https://api.example.com/items')
      .then(res => res.json())
      .then(data => {
        setData(data);
        setLoading(false);
      });
  }, []); // Empty dependency array ensures run on mount only
```

```
  if (loading) return <p>Loading...</p>;
  return <pre>{JSON.stringify(data, null, 2)}</pre>;
}
```

03 Advanced Hooks

- `useReducer`: An alternative to `useState` for complex state logic involving multiple sub-values or actions.
- `useMemo`: Memos and caches the calculated result of an expensive calculation across re-renders.
- `useCallback`: Caches a function definition between re-renders to prevent unnecessary child re-renders.
- `useRef`: Persists mutable values without triggering a re-render and directly accesses DOM nodes.

```
import React, { useReducer } from 'react';

function reducer(state, action) {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    default: return state;
  }
}

function CounterWithReducer() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <button onClick={() => dispatch({ type: 'increment' })}>
      Count: {state.count}
    </button>
  );
}
```

04 Custom Hooks

A **Custom Hook** is a reusable JavaScript function whose name starts with `use` and can call other Hooks. Custom Hooks allow you to extract component logic into reusable functions.

```
import { useState, useEffect } from 'react';

// Custom Hook for fetching data
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch(url)
```

```
      .then(res => res.json())
      .then(data => {
        setData(data);
        setLoading(false);
      });
  }, [url]);

  return { data, loading };
}
```